

Technisches Übergabedokument

Quellcode der Smarten Mülltonne

Raspberry Pi Pico W – Client-Server-Architektur

Projektgruppe Smarte Mülltonne

Stand: April 2026

Inhaltsverzeichnis

1	Architekturübersicht	4
1.1	Client-Server-Modell.....	4
1.2	Module und Dateien	4
1.3	Datenfluss	5
	Sensoren → GlobalController (Einlesen).....	5
	GlobalController → Aktoren (Ausgabe)	5
	Client ↔ Server (Netzwerkkommunikation).....	5
1.4	Zeitliche Ablaufsteuerung (Scheduling).....	5
2	Modulbeschreibungen.....	7
2.1	main.py – Einstiegspunkt und Konfiguration	7
	Konfigurationsparameter	7
	Aufbau der Hauptschleife	8
	Auskommentierter Testcode	8
2.2	global_controller.py – Zentrale Ablaufsteuerung.....	9
	Zustände des Roboters	9
	Aufbau eines Durchlaufs (run-Methode).....	9
	Weitere wichtige Methoden	10
	Hard-Offline-Modus	10
	Füllstand-Entprellung (Debouncing)	10
2.3	network_manager.py – Netzwerkverbindung	11
	Verbindungsablauf.....	11
	Wichtige Methoden	11
2.4	network_controller.py – Nachrichtenempfang	11
2.5	dcmotor.py – Motor-Treiber	13
	Funktionsprinzip.....	13
	Berechnung der Geschwindigkeit.....	13
	Optimierung	13
	Öffentliche Schnittstelle	13
2.6	PDcontroller.py – Regler für die Linienverfolgung.....	13
	Funktionsprinzip.....	13
	Sicherheitsgrenzen	14
2.7	liniensensor.py – Liniensensor-Array	15
	Positionsberechnung	15
	Störungsfilterung (Majority-Voting)	15
	Zwischenspeicherung (Caching)	15
	Bitmasken-Schnittstelle	15
2.8	ultraschallsensor.py – Ultraschall und Füllstand	16

HCSR04P – Abstandsmessung.....	16
FuellstandSensor – Prozentwert.....	16
2.9 led.py – RGB-LED-Steuerung	16
Pico-Status-LED (LEDStateMachine).....	16
Verbindungs-LED (ConnectionLEDStateMachine).....	17
2.10 buzzer.py – Akustische Signale	18
2.11 button.py – Taster-Verwaltung	18
2.12 mac_server.py – TCP-Server.....	18
2.13 mission_control.py – Missionslogik	20
Ablauf einer vollständigen Mission	20
Reconnect-Behandlung	20
Self-Healing (Statusabgleich)	20
3 Zentrale Abläufe.....	21
3.1 Linienverfolgung	21
3.2 Hinderniserkennung	21
3.3 Netzwerkkommunikation	21
Protokollübersicht	22
Bestätigungsmechanismus (Dreistufiger Handshake).....	22
3.4 Offline-Modus	22
4 Workarounds Motor-Asymmetrie	23
4.1 Übersicht aller Kompensationsstellen	23
4.2 Zusammenwirken der Maßnahmen.....	23
4.3 Empfehlung für die Folgegruppe	23
5 Erweiterbarkeit.....	24
5.1 Neue Sensoren hinzufügen.....	24
5.2 Neue Fahrmodi.....	24
5.3 Regelungsparameter anpassen	24
5.4 Serverprotokoll erweitern	24
5.5 Pin-Belegung.....	24

1 Architekturübersicht

1.1 Client-Server-Modell

Das System besteht aus zwei Rechnern, die über ein WLAN miteinander kommunizieren. In der Fachsprache wird dies als Client-Server-Architektur bezeichnet: Ein Rechner (der Client) stellt Anfragen oder sendet Daten, während der andere (der Server) diese entgegennimmt und darauf reagiert.

Client (Raspberry Pi Pico W): Der Pico ist der Bordcomputer der Mülltonne. Er liest alle Sensoren aus (Liniensensoren, Ultraschallsensoren, Buttons), steuert die Motoren, LEDs und den Buzzer und trifft sämtliche Fahrentscheidungen eigenständig. Die gesamte Fahrlogik läuft lokal auf dem Pico – er benötigt den Server nicht zum Fahren.

Server (Mac): Der Mac übernimmt die übergeordnete Einsatzplanung. Er entscheidet, wann die Mülltonne losfahren soll (z. B. wenn sie voll ist) und wann sie nach Hause zurückkehren soll (z. B. nach der Entleerung). Außerdem zeigt er den aktuellen Status in einer Konsole an.

Die Architektur ist bewusst so ausgelegt, dass der Pico auch ohne Server autonom fahren kann (Offline-Modus). Die Netzwerkverbindung ist also optional und dient primär der Fernsteuerung und Überwachung.

1.2 Module und Dateien

Das Projekt besteht aus 13 Python-Dateien, die jeweils eine klar abgegrenzte Aufgabe haben. Die folgende Tabelle gibt einen Überblick:

Seite	Datei	Aufgabe
Client (Pico)	main.py	Einstiegspunkt: Konfiguration, Hardware-Aufbau, Hauptschleife
Client	global_controller.py	Herzstück: Zentrale Ablaufsteuerung (Zustandsübergänge, Fahrlogik)
Client	network_manager.py	Kümmert sich um WLAN-Verbindung und TCP-Verbindung zum Server
Client	network_controller.py	Empfängt Nachrichten vom Server und leitet sie weiter
Client	dcmotor.py	Ansteuerung eines einzelnen DC-Motors
Client	PDcontroller.py	Regler für die Linienverfolgung (berechnet Lenkkorrektur)
Client	liniensensor.py	Liest die 5 Liniensensoren aus und berechnet die Linienposition
Client	ultraschallsensor.py	Ultraschall-Abstandsmessung und Füllstandsberechnung
Client	led.py	Steuerung der beiden RGB-LEDs (Status-Anzeige)
Client	buzzer.py	Abspielen von Signaltönen (ohne das Programm anzuhalten)
Client	button.py	Erkennung von kurzem und langem Tastendruck
Server (Mac)	mac_server.py	Nimmt Netzwerkverbindungen an, sendet und empfängt Nachrichten
Server	mission_control.py	Entscheidungslogik: Wann losfahren? Wann zurückkehren?

1.3 Datenfluss

Die Daten fließen in drei Richtungen durch das System. Dieser Abschnitt beschreibt, welche Information woher kommt, wie sie verarbeitet wird und wohin sie geht.

Sensoren → GlobalController (Einlesen)

Im regelmäßigen Takt von 20 ms liest der GlobalController die Sensordaten ein. Die Liniensensoren liefern eine Positionsangabe („wie weit ist die Linie von der Mitte entfernt?“). Der Hindernissensor misst den Abstand zu Gegenständen in Fahrtrichtung und der Füllstandssensor ermittelt, wie voll die Tonne ist. Zusätzlich werden die beiden Buttons abgefragt. All diese Informationen fließen als Eingangswerte in die zentrale Ablaufsteuerung ein und bestimmen, welche Aktionen ausgeführt werden.

GlobalController → Aktoren (Ausgabe)

Auf Basis der Sensordaten und des aktuellen Zustands berechnet der GlobalController Steuerbefehle für die Hardware. Der wichtigste Berechnungsschritt ist die Linienverfolgung: Ein Regler (PD-Regler, siehe Kapitel 2.6) berechnet, wie stark die Mülltonne nach links oder rechts lenken muss. Das Ergebnis wird als Geschwindigkeitswert an die beiden Motoren übergeben – der eine Motor wird schneller, der andere langsamer, wodurch eine Kurve entsteht. Gleichzeitig aktualisiert der GlobalController die LEDs (Farbe und Blinkmuster) und den Buzzer (Signaltöne), um den aktuellen Zustand für den Benutzer sicht- und hörbar zu machen.

Client ↔ Server (Netzwerkcommunication)

Die Kommunikation zwischen Pico und Mac erfolgt über eine TCP-Verbindung. TCP (Transmission Control Protocol) ist ein Netzwerkstandard, der sicherstellt, dass gesendete Nachrichten vollständig und in der richtigen Reihenfolge ankommen – ähnlich einem Einschreiben im Vergleich zu einer Postkarte.

Die Nachrichten sind einfache Textzeilen, jeweils mit einem Zeilenumbruch am Ende. Der Pico sendet regelmäßig seinen aktuellen Zustand an den Server (z. B. „STATUS:STANDBY“ oder „STATUS:LINE_FOLLOWING“). Der Server wertet diese aus und sendet bei Bedarf Fahrbefehle zurück (z. B. „GO_TO_STREET“). Eine vollständige Protokollübersicht findet sich in Kapitel 3.3.

1.4 Zeitliche Ablaufsteuerung (Scheduling)

Auf dem Pico gibt es kein Betriebssystem, das mehrere Programme gleichzeitig ausführt. Stattdessen läuft eine einzige Endlosschleife (Main-Loop), die abwechselnd verschiedene Aufgaben erledigt. Damit wichtige Aufgaben häufiger ausgeführt werden als weniger wichtige, nutzt das Programm zwei unterschiedliche Taktintervalle:

Aufgabe	Intervall	Frequenz	Was passiert
Fahren (Priorität 1)	20 ms	50-mal/s	Sensoren lesen, Regler berechnen, Motoren steuern
Netzwerk (Priorität 2)	50 ms	20-mal/s	Verbindung prüfen, Nachrichten senden/empfangen

Die Fahrt hat Vorrang, da Verzögerungen hier direkt spürbar wären (die Mülltonne würde von der Linie abkommen). Das Netzwerk wird seltener abgefragt, weil eine Verzögerung von 50 ms dort nicht auffällt.

Schutz gegen Verzögerungen: Wenn ein Durchlauf länger dauert als geplant (z. B. weil die automatische Speicherbereinigung aktiv war), werden verpasste Takte nicht nachgeholt, sondern übersprungen. Das Programm synchronisiert sich auf den aktuellen Zeitpunkt.

Zusätzlich wird die automatische Speicherbereinigung (Garbage Collection) so konfiguriert, dass sie seltener auslöst und die Fahrt nicht unterbricht.

2 Modulbeschreibungen

In diesem Kapitel wird jede Quellcode-Datei einzeln beschrieben: Zweck, enthaltene Klassen und Funktionen, Abhängigkeiten zu anderen Modulen sowie relevante Konfigurationsparameter.

2.1 main.py – Einstiegspunkt und Konfiguration

Diese Datei wird beim Einschalten des Picos automatisch ausgeführt. Sie enthält keine eigenen Klassen, sondern erfüllt drei Aufgaben: (1) alle Einstellungen als Konstanten festlegen, (2) die Hardware-Objekte erstellen und miteinander verknüpfen und (3) die Hauptschleife starten.

Konfigurationsparameter

Die wichtigsten Einstellungen sind als Konstanten am Anfang der Datei definiert. Bei Anpassungen am Verhalten der Mülltonne müssen in der Regel nur diese Werte geändert werden:

Parameter	Wert	Beschreibung
BASE_SPEED	40	Grundgeschwindigkeit der Motoren (Skala 0–100)
KP_WERT	0.6	Regelstärke: Wie stark reagiert die Lenkung auf Abweichungen
KD_WERT	0.02	Dämpfung: Verhindert, dass die Lenkung überschwingt
TRIM_MOTOR_A	1.1	Korrekturfaktor Motor A (links) – bekommt 10 % mehr Leistung
TRIM_MOTOR_B	1.0	Korrekturfaktor Motor B (rechts) – Standard
MINDUTY_A	29500	Mindest-Ansteuerungswert, ab dem Motor A anläuft
MINDUTY_B	27500	Mindest-Ansteuerungswert, ab dem Motor B anläuft
DRIVE_INTERVAL_MS	20	Zeitabstand zwischen zwei Fahr-Durchläufen (20 ms = 50 Hz)
NETWORK_INTERVAL_MS	50	Zeitabstand zwischen zwei Netzwerk-Durchläufen (50 ms = 20 Hz)
SERVER_IP	192.168.67.100	IP-Adresse des Mac-Servers im WLAN
SERVER_PORT	50002	Port für die Kommunikation (muss auf beiden Seiten gleich sein)
WLAN_SSID / PASS	iot-netz / 13248918	WLAN-Zugangsdaten (stehen im Klartext im Code)
PIVOT_LINE_COUNTER_ENABLED	False	Experimentelles Feature für die 180°-Drehung (deaktiviert)

Hinweis: Die WLAN-Zugangsdaten stehen im Klartext im Quellcode. Bei Änderung des Netzwerks müssen WLAN_SSID und WLAN_PASS direkt in main.py angepasst werden.

Aufbau der Hauptschleife

Am Ende von main.py befindet sich die Endlosschleife. Sie prüft bei jedem Durchlauf anhand von Zeitstempeln, ob der Fahr-Takt oder der Netzwerk-Takt fällig ist, und ruft die entsprechenden Funktionen auf. Ein Anti-Lag-Mechanismus verhindert, dass verpasste Takte nachgeholt werden: Stattdessen wird der nächste Tick auf die aktuelle Zeit gesetzt. Beim Beenden (z. B. durch Strg+C) werden im finally-Block alle Motoren gestoppt, der Buzzer ausgeschaltet, die LEDs deaktiviert und offene Netzwerkverbindungen geschlossen.

Auskommentierter Testcode

Am Ende der Datei befinden sich mehrere auskommentierte Testprogramme: ein Liniensensor-Test, eine einfache Geradeausfahrt und ein Ultraschall-Auslese-Programm. Diese Programme wurden während der Entwicklung zur Inbetriebnahme einzelner Komponenten verwendet und können bei Bedarf wieder aktiviert werden.

2.2 global_controller.py – Zentrale Ablaufsteuerung

Diese Datei ist mit ca. 1.350 Zeilen die umfangreichste des Projekts. Sie implementiert die gesamte Logik der Mülltonne als sogenannte Zustandsmaschine (State Machine). Das Grundprinzip ist: Der Roboter befindet sich immer in genau einem Zustand (z. B. „steht bereit“ oder „fährt auf der Linie“). Je nach aktuellem Zustand reagiert er unterschiedlich auf Ereignisse wie Sensordaten, Netzwerkbefehle oder Tastendruck. Zustände wechseln nur an definierten Übergängen – dadurch ist das Verhalten nachvollziehbar und testbar.

Zustände des Roboters

Zustand	Bedeutung
STANDBY	Ruheposition zuhause – wartet auf Befehl oder auf „Tonne voll“
LINE_FOLLOWING	Fährt aktiv auf der Linie
LINE_LOST	Linie verloren – fährt vorsichtig weiter und sucht (max. 10 s)
OBSTACLE	Hindernis erkannt – steht still und wartet, bis der Weg frei ist
USER_PAUSED	Manuell pausiert (roter Button gedrückt)
ARRIVED	Querlinie (Zielmarkierung) erkannt – 180°-Drehung läuft
WAIT_AT_STREET	Steht an der Straße und wartet auf Entleerung
FULL	Füllstand $\geq 95\%$ für mindestens 5 Sekunden bestätigt
EMPTIED	Füllstand $\leq 20\%$ für mindestens 5 Sekunden bestätigt
MANUAL_GOTO_STREET_REQUEST	Per Button wurde Start zur Straße angefordert
MANUAL_RETURN_HOME_REQUEST	Per Button wurde Rückfahrt angefordert
ERROR	Fehlerzustand – Motoren gestoppt

Die Zustände FULL und EMPTIED besitzen keinen eigenen Logik-Handler. Sie verhalten sich funktional wie STANDBY bzw. WAIT_AT_STREET (Motoren gestoppt, Sensoren werden gelesen), dienen aber als Signal an den Server, dass die Mülltonne voll bzw. entleert wurde.

Aufbau eines Durchlaufs (run-Methode)

Die Methode run() wird 50-mal pro Sekunde aus der Hauptschleife aufgerufen. Sie durchläuft bei jedem Aufruf vier Schritte in fester Reihenfolge:

1. Sensoren lesen (_step_sensors): Liest den Füllstandssensor aus. Dies geschieht nur, wenn die Mülltonne steht (Zustände STANDBY, WAIT_AT_STREET, FULL, EMPTIED und die manuellen Anforderungszustände). Während der Fahrt wird der Füllstandssensor nicht abgefragt, weil die Vibrationen die Messung verfälschen würden. In diesem Schritt wird auch die Füllstand-Überwachung ausgeführt: _handle_full_reporting() und _handle_empty_reporting() prüfen, ob die Schwellenwerte für „voll“ oder „entleert“ dauerhaft überschritten sind.

2. Hardware aktualisieren (_step_hardware): Lässt die LEDs blinken oder pulsieren und den Buzzer weiterlaufen. Diese Komponenten arbeiten zeitgesteuert und benötigen regelmäßige Aufrufe, damit ihre Muster (z. B. Doppelblitz oder Sinuspulsieren) flüssig ablaufen.

3. Eingaben prüfen (_step_input): Fragt die Buttons ab (kurz oder lang gedrückt?) und löst die entsprechende Aktion aus. Im Standby-Modus startet ein kurzer Druck auf Grün die

Fahrt, ein langer Druck erzwingt den Offline-Modus. Der rote Button pausiert die aktuelle Fahrt oder setzt bei langem Druck alles auf den Ausgangszustand zurück.

4. Zustandslogik ausführen (_step_logic): Verzweigt je nach aktuellem Zustand in die passende Logik-Methode. Im Zustand LINE_FOLLOWING wird die Linienvorgang berechnet und auf Hindernisse geprüft. Im Zustand OBSTACLE wird wiederholt gemessen, ob der Weg frei ist. Im Zustand ARRIVED wird die 180°-Drehung gesteuert. Ruhezustände (STANDBY, WAIT_AT_STREET) stoppen lediglich die Motoren.

Weitere wichtige Methoden

Methode	Aufgabe
follow_line()	Kernfunktion: Liest Liniensensoren, berechnet Lenkkorrektur, steuert Motoren (Details in Kapitel 3.1)
set_pico_state()	Wechselt den Zustand und löst Nebeneffekte aus: LED-Farbe, Buzzer-Ton, PD-Regler-Reset, Hard-Offline-Übergang
handle_network_command()	Verarbeitet eingehende Server-Befehle (GO_TO_STREET, RETURN_HOME, STOP) und leitet die Startsequenz ein
network_tick()	Wird im Netzwerk-Takt aufgerufen: Sendet Lebenszeichen (alle 2 s), arbeitet die Nachrichtenwarteschlange ab, steuert die Pre-Drive-Sequenz
_handle_full_reporting()	Überwacht den Füllstand im Standby. Nutzt eine 5-Sekunden-Entprellung mit Hysterese (Schwelle: 95 % voll, 90 % Reset)
_handle_empty_reporting()	Überwacht die Entleerung an der Straße (Schwelle: 20 % leer, 30 % Reset, 5 s Bestätigungszeit)
apply_speeds()	Setzt vorzeichenbehaftete Geschwindigkeiten (–100 bis +100) auf beide Motoren um (positiv = vorwärts, negativ = rückwärts)

Hard-Offline-Modus

Während der Fahrt wird die WLAN-Verbindung bewusst getrennt. Der Grund: Netzwerkaktivität beansprucht Rechenzeit, die während der Fahrt für die Motorsteuerung benötigt wird. Der Ablauf bei einem Server-Befehl ist:

(1) Bestätigung (ACK) an den Server senden. (2) Maximal 5 Sekunden auf die Gegenbestätigung (ACK_RECEIVED) warten. (3) Socket schließen (150 ms Wartezeit, damit die letzte Nachricht noch rausgeht). (4) WLAN-Netzwerkaktivität vollständig abschalten und losfahren. Nach Ankunft am Ziel wird die Verbindung automatisch über request_reconnect() wiederhergestellt.

Bei einem lokalen Start per Button (Offline-Modus) entfällt der ACK-Austausch. Die WLAN-Verbindung wird sofort getrennt und die Fahrt beginnt unmittelbar.

Füllstand-Entprellung (Debouncing)

Um Fehlauflösungen zu vermeiden (z. B. durch schwankende Messwerte beim Einwerfen von Müll), nutzt das System eine Entprellung mit Hysterese. Der Zustand FULL wird erst ausgelöst, wenn der Füllstand für mindestens 5 Sekunden ununterbrochen über 95 % liegt. Sobald er zwischendurch unter 95 % fällt, wird der Timer zurückgesetzt. Analog funktioniert die Entleerungserkennung (EMPTIED bei ≤ 20 % für 5 s). Zusätzlich wird ein Reset-Schwellenwert genutzt (Hysterese): Der „voll“-Status wird erst zurückgesetzt, wenn der Füllstand unter 90 % fällt, nicht schon bei 94 %.

2.3 network_manager.py – Netzwerkverbindung

Diese Datei verwaltet sowohl die WLAN-Verbindung zum Router als auch die TCP-Verbindung zum Server. Beide Verbindungstypen werden als separate Zustandsmaschinen implementiert und arbeiten vollständig automatisch: Der Pico baut die Verbindung selbstständig auf, erkennt Verbindungsabbrüche und verbindet sich automatisch neu.

Wichtiges Designprinzip – „Non-blocking“: Alle Netzwerkoperationen sind so programmiert, dass sie das Programm niemals anhalten. Wenn beispielsweise der WLAN-Verbindungsaufbau läuft, wartet der Pico nicht passiv darauf, sondern prüft bei jedem Durchlauf kurz den Status und macht sofort mit den anderen Aufgaben weiter. So kann der Roboter auch während eines Verbindungsversuchs weiterfahren.

Verbindungsablauf

Der Verbindungsaufbau erfolgt in zwei Stufen. Zuerst muss das WLAN zum Router hergestellt werden. Erst wenn das WLAN steht, beginnt der Aufbau der TCP-Verbindung zum Server. Für beide Stufen gelten Timeouts und Wiederholungsintervalle:

Parameter	Wert	Bedeutung
WIFI_TIMEOUT_MS	20 Sekunden	Maximale Dauer eines WLAN-Verbindungsversuchs
WIFI_RETRY_MS	2 Sekunden	Wartezeit vor einem erneuten WLAN-Versuch
SERVER_TIMEOUT_MS	8 Sekunden	Maximale Dauer eines TCP-Verbindungsversuchs
SERVER_RETRY_MS	2 Sekunden	Wartezeit vor einem erneuten Server-Versuch

Nach erfolgreichem Verbindungsaufbau sendet der Pico den Text „Pico ist bereit“ an den Server. Dieser erkennt daran, dass eine gültige Verbindung besteht. Zusätzlich wird das WLAN-Power-Management des Pico W deaktiviert (via `wlan.config(pm=0xa11140)`), um die Netzwerk-Latenz zu verbessern.

Wichtige Methoden

run(): Muss regelmäßig aufgerufen werden. Treibt beide Zustandsmaschinen (WLAN und TCP) voran.

intentional_disconnect(): Trennt die Verbindung auf Wunsch (Hard-Offline-Modus). Informiert den GlobalController nicht über einen Fehler, da die Trennung beabsichtigt ist.

request_reconnect(): Erzwingt einen sofortigen Wiederverbindungsversuch, z. B. nach Ende einer Fahrt. Setzt die Retry-Timer zurück.

mark_server_disconnected(): Wird bei einem Fehler auf dem Socket aufgerufen. Schließt das Socket, benachrichtigt den GlobalController und löst die automatische Reconnect-Logik aus.

2.4 network_controller.py – Nachrichtenempfang

Während `network_manager.py` für den Verbindungsaufbau zuständig ist, kümmert sich der `NetworkController` um das Empfangen und Aufbereiten eingehender Nachrichten.

Daten kommen über das Netzwerk nicht unbedingt als vollständige Nachrichten an. Ein Datenpaket kann mitten in einem Satz enden oder mehrere Nachrichten auf einmal enthalten. Der `NetworkController` sammelt daher alle eingehenden Daten in einem Zwischenspeicher (Buffer) und sucht nach Zeilenumbrüchen als Trennzeichen. Erst wenn

eine vollständige Zeile vorliegt, wird sie als Befehl an `global_controller.handle_network_command()` übergeben.

Socket-Verwaltung: Falls noch kein Socket vorhanden ist, holt sich der NetworkController bei jedem Aufruf automatisch das aktuelle Socket vom NetworkManager. Bei einem Verbindungsabbruch werden sowohl der GlobalController (für die LED-Anzeige) als auch der NetworkManager (für den automatischen Reconnect) informiert.

Buffer-Overflow-Schutz: Der Zwischenspeicher wird bei Überschreitung von 2.048 Bytes auf 1.024 Bytes gekürzt, um den begrenzten Arbeitsspeicher des Picos zu schützen.

MicroPython-Workaround: Auch wenn das Polling-System keine eingehenden Daten meldet, wird sicherheitshalber einmal eine Empfangsoperation versucht. Der Grund: Manche MicroPython-Versionen melden verfügbare Daten nicht zuverlässig über den Polling-Mechanismus.

2.5 dcmotor.py – Motor-Treiber

Diese Datei stellt die Klasse DCMotor bereit, die einen einzelnen DC-Motor ansteuert. Die Motoren sind über eine sogenannte H-Brücke (konkret ein L298N-Modul) mit dem Pico verbunden. Die H-Brücke ist ein Elektronikbauteil, das es ermöglicht, die Drehrichtung des Motors umzukehren, indem die Polarität der Stromversorgung vertauscht wird.

Funktionsprinzip

Jeder Motor wird über drei Verbindungen gesteuert. Zwei Richtungspins bestimmen, ob der Motor vorwärts oder rückwärts dreht: Ist Pin 1 auf HIGH und Pin 2 auf LOW, dreht der Motor vorwärts; umgekehrt dreht er rückwärts. Der dritte Pin steuert die Geschwindigkeit über ein sogenanntes PWM-Signal (Pulsweitenmodulation). Dabei wird der Pin sehr schnell ein- und ausgeschaltet – je länger die „Ein“-Phase im Verhältnis zur „Aus“-Phase ist, desto schneller dreht der Motor. Der PWM-Wert (genannt Duty Cycle) reicht von 0 (Motor steht) bis 65.535 (Motor dreht maximal schnell).

Berechnung der Geschwindigkeit

Die gewünschte Geschwindigkeit (0–100 %) wird in einen PWM-Wert umgerechnet. Dabei wird berücksichtigt, dass jeder Motor eine sogenannte Totzone hat: Unterhalb eines bestimmten Mindestwerts (MINDUTY) dreht sich der Motor nicht, weil die elektrische Energie nicht ausreicht, um die mechanische Reibung zu überwinden.

Zusätzlich wird ein Korrekturfaktor (trim_factor) angewendet, der die Geschwindigkeit proportional skaliert. Dieser Faktor kann zur Laufzeit dynamisch geändert werden und dient dem Ausgleich der Motorasymmetrie (siehe Kapitel 4).

Optimierung

Um unnötige Zugriffe auf die Hardware zu vermeiden, merkt sich die Klasse den zuletzt gesetzten Befehl (current_speed und current_direction). Wenn derselbe Befehl erneut kommt (z. B. „vorwärts mit 40 %“), wird er nicht erneut an die Pins geschrieben. Das spart Rechenzeit, da PWM-Schreibzugriffe vergleichsweise teuer sind.

Öffentliche Schnittstelle

forward(speed): Fährt vorwärts mit der angegebenen Geschwindigkeit (0–100).

backward(speed): Fährt rückwärts.

stop(): Stoppt den Motor sofort (Duty Cycle auf 0, beide Richtungspins auf LOW).

2.6 PDcontroller.py – Regler für die Linienverfolgung

Der PD-Regler ist das mathematische Herzstück der Linienverfolgung. Er berechnet anhand der aktuellen Sensorposition, wie stark die Mülltonne nach links oder rechts lenken muss, um auf der Linie zu bleiben.

Funktionsprinzip

Der Regler kombiniert zwei Berechnungskomponenten zu einem einzigen Korrekturwert:

P-Anteil (Proportional): Je weiter die Linie von der Mitte abweicht, desto stärker wird gegengelenkt. Beispiel: Linie ist weit rechts → stark nach rechts lenken. Dies ist die Hauptkomponente der Lenkung und sorgt dafür, dass der Roboter grundsätzlich der Linie folgt.

D-Anteil (Differenzial): Berücksichtigt, wie schnell sich die Abweichung ändert. Wenn die Linie plötzlich schnell nach rechts wandert, bremst der D-Anteil die Reaktion ab, damit die Mülltonne nicht über das Ziel hinauschießt und zu „wackeln“ beginnt. Er wirkt wie ein Dämpfer gegen Schwingungen.

Ein I-Anteil (Integral), wie man ihn aus der klassischen PID-Regelung kennt, wurde bewusst weggelassen. Der Grund: Die Liniensensoren liefern nur digitale Werte (0 oder 1), keine kontinuierlichen Messwerte. Ein Integrator würde bei diesen sprunghaften Werten zu Instabilitäten führen.

Die Berechnung kombiniert beide Anteile:

$$\text{Korrektur} = \text{Abweichung} \times \text{KP} + \text{Änderungsrate} \times \text{KD}$$

Diese Korrektur wird anschließend auf die Motorgeschwindigkeiten aufgeteilt: linker Motor = Grundgeschwindigkeit – Korrektur, rechter Motor = Grundgeschwindigkeit + Korrektur.

Sicherheitsgrenzen

Maximale Korrektur (55): Begrenzt den Ausgang des Reglers, damit die Motoren nicht in extreme Bereiche gesteuert werden. Ohne diese Begrenzung könnte ein sehr großer Fehler dazu führen, dass ein Motor weit über 100 % angesteuert wird.

Maximale Änderungsrate (2.000/s): Filtert Messausreißer, die durch Störungen an den Sensoren entstehen können. Ohne diesen Filter würde ein einzelner fehlerhafter Messwert zu einer abrupten Lenkänderung führen.

Reset-Funktion: Setzt den internen Zustand des Reglers zurück (letzter Fehler und Zeitstempel). Wird automatisch aufgerufen, wenn die Linie verloren oder der Roboter gestoppt wird, damit beim Neustart keine alten Werte nachwirken.

2.7 liniensensor.py – Liniensensor-Array

Verwaltet die fünf digitalen Liniensensoren, die nebeneinander unter dem Fahrzeug montiert sind. Jeder Sensor erkennt, ob sich unter ihm eine schwarze Linie auf hellem Untergrund befindet (Wert 1) oder nicht (Wert 0). Aus der Kombination aller fünf Werte berechnet die Klasse eine gewichtete Position der Linie.

Positionsberechnung

Jeder Sensor hat ein Gewicht, das seiner physischen Position entspricht. Der äußerste linke Sensor hat das Gewicht -100 , der mittlere 0 und der äußerste rechte $+100$. Die Position wird als gewichteter Durchschnitt aller aktiven Sensoren berechnet. Beispiel: Wenn nur der mittlere Sensor aktiv ist, ist die Position 0 (mittig). Wenn der linke Außensensor aktiv ist, ist die Position -100 (Linie ganz links). Sind mehrere benachbarte Sensoren aktiv, ergibt sich ein Zwischenwert.

Sensor	GPIO	Rohgewicht	Effektiver Faktor
Links Außen (LA)	8	-100	1.0
Links Mitte (LM)	9	-50	1.5 (Asymmetrie-Workaround, siehe Kapitel 4)
Mitte (M)	10	0	1.0
Rechts Mitte (RM)	11	$+50$	1.0
Rechts Außen (RA)	12	$+100$	1.0

Der Faktor 1.5 beim linken Mittel-Sensor ist ein Workaround für die Motor-Asymmetrie. Er bewirkt, dass die berechnete Position bei Abweichungen nach links stärker ausschlägt. Der Regler korrigiert dadurch entschiedener nach links, was die schwächere Linkslenkung kompensiert.

Wenn kein einziger Sensor eine Linie erkennt (alle Werte = 0), gibt die Methode `None` zurück. Die aufrufende Logik im `GlobalController` erkennt daran, dass die Linie verloren wurde.

Störungsfilterung (Majority-Voting)

Jeder Sensor wird standardmäßig dreimal hintereinander abgetastet (konfigurierbar über `samples_per_read`). Zwischen den Abtastungen liegt eine Pause von 80 Mikrosekunden. Es zählt der Mehrheitswert: Wenn zwei von drei Messungen „1“ ergeben, wird „1“ übernommen. Dies filtert kurze elektrische Störimpulse heraus, die bei den digitalen Sensoren gelegentlich auftreten und sonst zu Fehlreaktionen der Lenkung führen könnten.

Zwischenspeicherung (Caching)

Die berechnete Position wird für mindestens 8 ms zwischengespeichert. Wenn `get_position()` innerhalb dieses Zeitfensters erneut aufgerufen wird, liefert sie den gespeicherten Wert, ohne die Sensoren erneut auszulesen. Dies spart Rechenzeit, da die Sensoren in mehreren Programmteilen abgefragt werden.

Bitmasken-Schnittstelle

Neben der berechneten Position speichert die Klasse auch das rohe Bitmuster der Sensoren (`_cached_bits`). Dieses wird vom `GlobalController` für Spezialfälle ausgelesen, z. B. um zu erkennen, ob nur der äußerste rechte Sensor aktiv ist (Bit-Muster `0b00001`).

2.8 ultraschallsensor.py – Ultraschall und Füllstand

Diese Datei enthält zwei Klassen: HCSR04P als Treiber für den physischen Sensor und FuellstandSensor als Interpretationsschicht, die gemessene Distanzen in Füllstandsprozent umrechnet.

HCSR04P – Abstandsmessung

Der Ultraschallsensor funktioniert nach dem Echolot-Prinzip: Er sendet einen kurzen Schallimpuls aus und misst die Zeit, bis das Echo zurückkommt. Aus der Laufzeit ergibt sich der Abstand zum nächsten Gegenstand (Formel: Distanz in cm = Laufzeit in Mikrosekunden / 58).

Die Besonderheit dieser Implementierung ist, dass die Messung im Hintergrund abläuft, ohne das Programm anzuhalten. Der Pico löst den Schallimpuls aus und registriert dann über einen sogenannten **Interrupt** (eine automatische Benachrichtigung der Hardware), wann das Echo eintrifft. Zwischen Auslösen und Empfang kann der Pico andere Aufgaben erledigen. Ein Interrupt für die steigende Flanke (Echo beginnt) speichert den Startzeitpunkt, ein zweiter für die fallende Flanke (Echo endet) berechnet die Distanz. Kommt nach 30 ms kein Echo, gilt die Messung als fehlgeschlagen (Timeout).

Im Projekt werden zwei Instanzen dieser Klasse verwendet: Eine für den Füllstand (Messintervall 250 ms) und eine für die Hinderniserkennung (Messintervall 100 ms). Die unterschiedlichen Intervalle spiegeln die unterschiedliche Dringlichkeit wider: Hindernisse müssen schneller erkannt werden als Füllstandsänderungen.

FuellstandSensor – Prozentwert

Wandelt den gemessenen Abstand in einen Füllstand um. Dazu werden zwei kalibrierte Referenzwerte verwendet:

Parameter	Wert	Bedeutung
leer_abstand_cm	27,85	Abstand vom Sensor zum Tonnenboden (= 0 % Füllstand)
voll_abstand_cm	10,0	Abstand vom Sensor zum Müll bei voller Tonne (= 100 %)
deckel_offen_margin_cm	5,0	Toleranz für die Erkennung eines offenen Deckels

Der Füllstand wird linear zwischen diesen beiden Werten berechnet. Ist der gemessene Abstand z. B. 18,9 cm, ergibt sich: $(27,85 - 18,9) / (27,85 - 10,0) = \text{ca. } 50 \%$.

Deckel-Offen-Erkennung: Drei aufeinanderfolgende Timeouts (kein Echo) oder ein Abstand größer als leer_abstand + margin (32,85 cm) werden als offener Deckel interpretiert. In diesem Fall wird kein Füllstandswert berechnet, um Fehlmessungen zu vermeiden.

2.9 led.py – RGB-LED-Steuerung

Enthält drei Klassen für die beiden RGB-LEDs des Roboters. Die Basisklasse LED steuert die Farbe über PWM (Mischung aus Rot, Grün und Blau mit jeweils 0–65.535 Helligkeitsstufen). Die beiden Zustandsmaschinen LEDStateMachine und ConnectionLEDStateMachine erzeugen Effekte wie Blinken, Pulsieren oder Doppelblitzen, ohne das Programm anzuhalten.

Pico-Status-LED (LEDStateMachine)

Zeigt den aktuellen Betriebszustand des Roboters an. Die Zuordnung ist:

Zustand	Effekt	Farbe
STANDBY	Dauerlicht	Grün
FULL	Dauerlicht	Gelb
LINE_FOLLOWING / LINE_LOST	Sanftes Auf- und Abdimmen (Sinus-Pulsieren)	Blau
USER_PAUSED	Dauerlicht	Blau
WAIT_AT_STREET	Pulsieren	Gelb
EMPTIED	Pulsieren	Grün
OBSTACLE	Doppelblitz (zweifaches kurzes Aufblitzen)	Orange
ARRIVED	Pulsieren	Blau

Zusätzlich gibt es ein Pending-Overlay: Wenn der Roboter auf eine Bestätigung wartet (z. B. nach einem Button-Druck), überlagert ein schnelles Blinken die normale Anzeige. Gelbes Blinken signalisiert „voll“, grünes Blinken signalisiert „leer“.

Verbindungs-LED (ConnectionLEDStateMachine)

Zeigt den Netzwerkstatus an:

Zustand	Anzeige
Verbunden	Grün Dauerlicht
Verbindungsaufbau	Blau schnell blinkend (alle 150 ms)
Nicht verbunden	Blau langsam blinkend (alle 700 ms)
Offline-Modus	Lila Dauerlicht

2.10 buzzer.py – Akustische Signale

Die Klasse Buzzer steuert einen einfachen Piezo-Signalgeber, der nur zwei Zustände kennt: an und aus (kein Ton mit variabler Frequenz). Trotz dieser Einfachheit ermöglicht die Klasse komplexe Signalmuster, indem sie An- und Aus-Phasen mit unterschiedlichen Dauern kombiniert.

Muster-Definition: Ein Tonmuster (Pattern) ist eine Liste von Tupeln, z. B. [(True, 100), (False, 100), (True, 100), (False, 1)] für einen Doppelton. Jeder Eintrag besteht aus einem Zustand (True = an, False = aus) und einer Dauer in Millisekunden.

Wiederholung: Muster können optional wiederholt werden. Die Pause zwischen Wiederholungen variiert zufällig innerhalb eines einstellbaren Bereichs (z. B. zwischen 2 und 3 Sekunden). Diese Variation macht den Hindernis-Alarm weniger eintönig.

Active-Low-Konfiguration: Der Buzzer in dieser Hardware ist Active Low, d. h. er erzeugt Ton, wenn der Pin auf niedrigen Pegel (0) gesetzt wird, und ist still bei hohem Pegel (1). Die Klasse abstrahiert dies über den Parameter `active_high=False`, sodass im Code einfach `on()` und `off()` aufgerufen werden kann. Im Cleanup-Block von `main.py` wird der Pin 13 explizit auf HIGH gesetzt, um den Buzzer sicher stumm zu schalten.

Zustandswechsel-Signale: Der GlobalController nutzt den Buzzer in `_handle_buzzer_state_change()` für verschiedene Situationen: Ein langer Ton beim Losfahren, ein kurzer Ton beim Pausieren, ein Doppelton-Alarm bei Hindernissen (mit Wiederholung) und ein mittlerer Ton bei Ankunft.

2.11 button.py – Taster-Verwaltung

Die Klasse Button verwaltet einen physischen Taster und erkennt sowohl kurze als auch lange Tastendrücke. Im Projekt gibt es zwei Buttons: Rot (GPIO 0) für Stopp/Reset und Grün (GPIO 1) für Start/Bestätigung.

Funktionsprinzip: Die Buttons sind als „Active Low“ konfiguriert, d. h. sie sind über interne Pull-Up-Widerstände mit dem Pico verbunden. Im Ruhezustand liest der Pin den Wert 1 (HIGH). Beim Drücken wird der Kontakt gegen Masse (GND) geschlossen und der Pin liest 0 (LOW).

Drucktyp-Erkennung: Die Methode `check_press_type()` wird in jedem Durchlauf der Hauptschleife aufgerufen. Sie erkennt den Zeitpunkt des Drückens und des Loslassens und berechnet die Dauer. Ein Druck zwischen 50 und 1.000 ms wird als SHORT gewertet, über 1.000 ms als LONG. Die Erkennung erfolgt erst beim Loslassen, da erst dann die Dauer feststeht.

Entprellung (Debouncing): Mechanische Taster erzeugen beim Drücken und Loslassen kurzzeitig wackelnde Signale (Prellen). Eine Software-Entprellung von 50 ms stellt sicher, dass jeder Tastendruck nur einmal erkannt wird.

2.12 mac_server.py – TCP-Server

Diese Datei läuft auf dem Mac und bildet die Gegenstelle zum Pico. Der Server wartet auf eingehende Verbindungen und verarbeitet jede Verbindung in einem separaten Thread (einem parallel laufenden Programmstrang). So könnte theoretisch mehr als ein Pico gleichzeitig verbunden sein.

Ablauf pro Verbindung: Der Server führt in einer Schleife folgende Schritte aus: (1) Er prüft, ob neue Daten vom Pico vorliegen, wobei er maximal 1 Sekunde wartet. (2) Vollständige Nachrichten (erkennbar am Zeilenumbruch) werden an die Missionslogik (MissionControl) weitergereicht. (3) Direkte Antworten wie Bestätigungen werden sofort zurückgesendet. (4) Es wird geprüft, ob die Missionslogik einen Befehl senden möchte (z. B. „Fahre zur Straße“). (5) Timer werden geprüft (z. B. ob eine Bestätigung überfällig ist).

Inaktivitäts-Timeout: Eine Verbindung wird nach 5 Minuten (300 Sekunden) ohne eingehende Daten automatisch getrennt.

Logging: Alle relevanten Ereignisse werden mit Zeitstempel und Kategorie auf der Konsole ausgegeben. Die Formatierung nutzt Trennlinien für wichtige Abschnitte (z. B. neue Verbindung) und einzeilige Meldungen für laufende Kommunikation.

2.13 mission_control.py – Missionslogik

Enthält die serverseitige Entscheidungslogik. Diese Klasse reagiert auf Statusmeldungen des Picos und steuert den Ablauf einer vollständigen Einsatzmission (Hinfahrt, Entleerung, Rückfahrt).

Ablauf einer vollständigen Mission

- 1. Überwachung (MONITORING):** Der Server empfängt laufend Status-Updates vom Pico und wartet auf das Ereignis „Tonne voll“ (STATUS:FULL) oder einen manuellen Startbefehl per Button (STATUS:MANUAL_GOTO_STREET_REQUEST).
- 2. Planung (PLANING):** Sobald eines dieser Ereignisse eintritt, startet ein 10-Sekunden-Timer. Diese Verzögerung gibt dem Benutzer Zeit, den Vorgang auf der Konsole zu beobachten, bevor die Mülltonne losfährt. Die 10 Sekunden sind ein Demo-Feature und können in `check_timeouts()` angepasst werden.
- 3. Warten auf Bestätigung (WAITING_FOR_ACK):** Nach Ablauf des Timers wird der Fahrbefehl gesendet. Der Server wartet darauf, dass der Pico ihn bestätigt (ACK). Bleibt die Bestätigung länger als 5 Sekunden aus, wird der Befehl automatisch erneut gesendet.
- 4. Missionsüberwachung (MISSION_MONITORING):** Während der Fahrt überwacht der Server den Pico-Status. Da der Pico im Hard-Offline-Modus fährt, ist während der Fahrt in der Regel keine Verbindung aktiv. Der Server erwartet eine Meldung nach Ankunft.
- 5. Warten auf Entleerung (WAITING_FOR_EMPTYING):** Nach der Meldung ARRIVED: STREET wartet der Server, bis der Pico meldet, dass die Tonne entleert wurde (STATUS:EMPTIED) oder ein manueller Rückruf erfolgt. Dann beginnt der Ablauf erneut mit einem 10-Sekunden-Timer, gefolgt vom Rückfahrt-Befehl.

Reconnect-Behandlung

Wenn die Verbindung zum Pico unterbrochen und später wiederhergestellt wird, muss der Server den korrekten Zustand wiederherstellen, ohne Befehle doppelt zu senden. Dazu nutzt er zwei gespeicherte Informationen: den zuletzt gemeldeten Pico-Status (`_last_seen_pico_state`, z. B. `USER_PAUSED` oder `OBSTACLE`) und den letzten bekannten Ankunftsort (`_last_arrived_place`: `HOME` oder `STREET`). Aus diesen Informationen leitet er ab, ob der Pico zuhause ist (weiter überwachen), an der Straße steht (auf Entleerung warten) oder ob der Status mehrdeutig ist (abwarten).

Self-Healing (Statusabgleich)

Die Methode `_sync_state_with_pico()` gleicht den Server-Zustand laufend mit den Heartbeat-Meldungen des Picos ab. Wenn der Server z. B. denkt, er sei im Zustand `MONITORING`, der Pico aber `LINE_FOLLOWING` meldet, korrigiert der Server seinen Zustand auf `MISSION_MONITORING`. Dies fängt Situationen ab, in denen der Server z. B. neugestartet wurde, während der Pico noch unterwegs war.

3 Zentrale Abläufe

Dieses Kapitel beschreibt die wichtigsten Funktionsabläufe des Systems im Zusammenspiel der Module.

3.1 Linienverfolgung

Die Linienverfolgung ist die Kernfunktion des Roboters. Sie wird in `global_controller.follow_line()` implementiert und bei jedem Fahr-Tick (alle 20 ms) aufgerufen.

Schritt 1 – Sensorabfrage: `liniensensor.get_position()` liefert einen Wert zwischen -100 (Linie ganz links) und +100 (Linie ganz rechts). Der Wert 0 bedeutet „Linie ist genau in der Mitte“. Wird keine Linie erkannt, liefert die Funktion None.

Schritt 2 – Linienverlust-Toleranz: Kurze Lücken in der Linie (z. B. durch Knicke oder Verschmutzungen) werden toleriert. Das Programm zählt mit, wie viele aufeinanderfolgende Messungen keine Linie ergeben haben. Erst wenn innerhalb von 800 ms mindestens 12 solcher Messungen auftreten, wechselt der Zustand zu `LINE_LOST`. In der Zwischenzeit wird die letzte bekannte Position als Schätzwert verwendet.

Schritt 3 – Reglerberechnung: Der PD-Regler berechnet eine Korrektur. Die Motorgeschwindigkeiten ergeben sich aus: linker Motor = Grundgeschwindigkeit – Korrektur, rechter Motor = Grundgeschwindigkeit + Korrektur. Ist die Linie z. B. links, wird die Korrektur positiv, der rechte Motor wird schneller und die Mülltonne lenkt nach links.

Schritt 4 – Kurvenhilfe: Bei engen Kurven reicht die normale Regelung nicht aus. Wenn die Geschwindigkeit des rechten Motors unter 5 fällt, wird eine Drehung auf der Stelle eingeleitet: linker Motor voll vorwärts (100), rechter Motor rückwärts (-20). Bei bestimmten Sensorkombinationen wird zusätzlich der Korrekturfaktor des linken Motors temporär erhöht (siehe Kapitel 4).

Schritt 5 – Motoransteuerung: Die berechneten Geschwindigkeiten werden auf den Bereich -100 bis +100 begrenzt und an `apply_speeds()` übergeben, das Vorwärts- und Rückwärtsbefehle an die Motoren sendet.

3.2 Hinderniserkennung

Der vordere Ultraschallsensor prüft bei jedem Fahr-Tick den Abstand zum nächsten Gegenstand. Die Erkennung arbeitet mit zwei unterschiedlichen Schwellenwerten, um ein Hin-und-Her-Schalten zu vermeiden (Hysteresis-Prinzip):

Erkennung (unter 30 cm): Sobald ein Gegenstand näher als 30 cm erkannt wird, wechselt die Mülltonne sofort in den Zustand `OBSTACLE` und stoppt die Motoren. Der vorherige Zustand wird gespeichert, damit nach der Freigabe nahtlos weitergefahren werden kann.

Freigabe (30 Messungen über 35 cm): Der Weg gilt erst dann als frei, wenn 30 aufeinanderfolgende Messungen einen Abstand von über 35 cm ergeben. Diese Entprellung verhindert, dass die Mülltonne bei schwankenden Messwerten ständig anhält und losfährt. Der höhere Schwellenwert für die Freigabe (35 statt 30 cm) stellt sicher, dass das Hindernis wirklich weg ist, nicht nur geringfügig weiter entfernt.

3.3 Netzwerkkommunikation

Die Kommunikation zwischen Pico und Server basiert auf einem einfachen Textprotokoll. Jede Nachricht ist eine einzelne Textzeile, abgeschlossen mit einem Zeilenumbruch.

Protokollübersicht

Richtung	Nachricht	Bedeutung
Pico → Server	Pico ist bereit	Begrüßung nach Verbindungsaufbau
Pico → Server	STATUS:<Zustand>	Regelmäßiges Lebenszeichen mit aktuellem Zustand (alle 2 s)
Pico → Server	ACK CMD_GOTO_STREET	Bestätigung: Fahrbefehl wurde angenommen
Pico → Server	ACK CMD_RETURN_HOME	Bestätigung: Rückfahrt-Befehl wurde angenommen
Pico → Server	ARRIVED: STREET	Meldung: Bin an der Straße angekommen
Pico → Server	ARRIVED: HOME	Meldung: Bin zuhause angekommen
Server → Pico	GO_TO_STREET	Befehl: Fahre zur Straße
Server → Pico	RETURN_HOME	Befehl: Fahre nach Hause
Server → Pico	STOP	Befehl: Sofort anhalten
Server → Pico	ACK_RECEIVED CMD_...	Gegenbestätigung: Bestätigung ist angekommen

Bestätigungsmechanismus (Dreistufiger Handshake)

Um sicherzustellen, dass Befehle angekommen sind, nutzt das System einen dreistufigen Austausch: (1) Server sendet den Befehl (z. B. GO_TO_STREET). (2) Pico bestätigt mit ACK CMD_GOTO_STREET. (3) Server bestätigt die Bestätigung mit ACK_RECEIVED CMD_GOTO_STREET. Erst nach diesem vollständigen Austausch trennt der Pico die Verbindung und fährt los. Wenn Schritt 3 ausbleibt (z. B. wegen Netzwerkproblemen), fährt der Pico nach einem Timeout von 5 Sekunden trotzdem los.

3.4 Offline-Modus

Das System kennt drei verschiedene Offline-Szenarien:

Automatischer Offline-Modus: Wenn der Server nach 30 Sekunden nicht erreichbar ist (connection_timeout_ms), wechselt der Pico automatisch in den Offline-Modus. Die Verbindungs-LED zeigt Lila. Der Pico akzeptiert weiterhin Button-Eingaben für lokale Starts. Im Hintergrund wird weiterhin versucht eine Verbindung aufzubauen. Bei erfolgreichem Verbindungsaufbau, wechselt der Pico wieder in den Online-Betrieb.

Hard-Offline (während der Fahrt): Beim Start einer Fahrt wird die WLAN-Verbindung proaktiv getrennt. Netzwerk-Ticks werden komplett übersprungen (is_network_allowed() gibt False zurück). Nach Ankunft am Ziel wird die Verbindung automatisch wiederhergestellt.

Manueller Offline (langer Druck auf Grün): Erzwingt den Offline-Modus, wenn keine Verbindung besteht. Nützlich, wenn man nicht auf den automatischen Timeout (30 s) warten möchte.

4 Workarounds Motor-Asymmetrie

Die beiden DC-Motoren der Mülltonne reagieren nicht gleich auf Steuersignale: Motor A (links) verhält sich nicht entsprechend den gesetzten Befehlen. Motor B (rechts) funktioniert hingegen problemlos. Ohne Gegenmaßnahmen würde die Mülltonne bei einer Rechtskurve aus der Kurve fahren, da der berechnete, hohe Korrekturwert nicht dafür sorgt, dass Motor A ausreichend korrigiert wird. Im Code sind insgesamt vier Kompensationsmaßnahmen implementiert, verteilt über drei Dateien.

4.1 Übersicht aller Kompensationsstellen

Nr.	Datei	Zeile(n)	Maßnahme	Erklärung
1	main.py	28–29	TRIM_MOTOR_A = 1.1, TRIM_MOTOR_B = 1.0	Startwert: Motor A bekommt 10 % mehr Leistung. Wirkt beim Geradeausfahren und bei der 180°-Drehung, nicht während der Linienverfolgung.
2	main.py	54–55	MINDUTY_A = 29500, MINDUTY_B = 27500	Unterschiedliche Totzonen: Motor A braucht einen höheren Anfangswert, um überhaupt anzulaufen.
3	global_controller.py	1097–1099	Wenn nur RA aktiv: trim = 1.2, left = 100	Spezialfall bei enger Rechtskurve: Motor A wird temporär geboostet.
4	liniensensor.py	168	Links-Mitte-Gewicht × 1.5	Verstärkt die berechnete Linienposition nach links.

4.2 Zusammenwirken der Maßnahmen

Die vier Kompensationen wirken auf unterschiedlichen Ebenen und ergänzen sich:

Hardware-Ebene (Nr. 2 – MINDUTY): Kompensiert die unterschiedliche Anlaufschwelle der Motoren. Diese Werte wurden experimentell ermittelt, indem der PWM-Wert schrittweise erhöht wurde, bis sich der Motor gerade eben dreht.

Sensor-Ebene (Nr. 4 – Gewichtungsfaktor): Verschiebt die von den Sensoren berechnete Position. Der Regler „denkt“, die Linie sei weiter links als sie tatsächlich ist, und lenkt entsprechend stärker nach links.

Regler-Ebene (Nr. 1, 3 – Trim-Faktoren): Skalieren die Motorleistung direkt. Im Spezialfall (nur äußerster rechter Sensor aktiv) wird der Trim temporär auf 1.2 erhöht, um enge Rechtskurven zu unterstützen.

4.3 Empfehlung für die Folgegruppe

Sollten die Motoren ausgetauscht oder die Mechanik verändert werden, müssen alle fünf Stellen geprüft und ggf. angepasst werden. Die empfohlene Reihenfolge ist: (1) MINDUTY-Werte experimentell neu bestimmen. (2) Trim-Faktoren bei Geradeausfahrt kalibrieren. (3) Gewichtungsfaktor in der Positionsberechnung prüfen. Bei symmetrischen Motoren sollten alle Kompensationswerte auf ihre Neutralwerte zurückgesetzt werden (Trim = 1.0, Gewichtungsfaktor = 1.0, gleiche MINDUTY-Werte).

5 Erweiterbarkeit

Die modulare Architektur ermöglicht gezielte Erweiterungen, ohne das Gesamtsystem umbauen zu müssen.

5.1 Neue Sensoren hinzufügen

Neue Sensoren lassen sich in vier Schritten integrieren:

- 1. Treiberklasse erstellen:** Orientierung an `ultraschallsensor.py` oder `liniensensor.py`. Wichtig: Die Klasse muss eine `run()`-Methode haben, die ohne Wartezeiten arbeitet (bei jedem Aufruf kurz prüfen, ob Daten vorliegen, und sofort zurückkehren).
- 2. In `main.py` instanziiieren:** Pin-Belegung konfigurieren und das Sensor-Objekt erstellen.
- 3. An `GlobalController` übergeben:** Entweder als neuer Konstruktor-Parameter oder über eine `set_*`-Methode (analog zu `set_fuellstand_sensor()`).
- 4. In `_step_sensors()` einbinden:** Die `run()`-Methode des Sensors aufrufen und die Ergebnisse in die Logik einbinden.

5.2 Neue Fahrmodi

Neuen Zustand definieren: In `global_controller.py` eine neue Konstante anlegen (z. B. `STATE_OBSTACLE_AVOIDANCE`).

Logik-Handler schreiben: Methode `_logic_obstacle_avoidance()` implementieren und in `_step_logic()` einhängen.

LED und Buzzer: In `led.py` das Zustand-Farb-Mapping ergänzen. In `_handle_buzzer_state_change()` ein Tonmuster definieren.

Server informieren: In `mission_control.py` die Übersetzungstabelle und `_sync_state_with_pico()` erweitern.

5.3 Regelungsparameter anpassen

KP_WERT und KD_WERT: Bestimmen das Lenkverhalten. Höheres KP = stärkere Reaktion, aber mehr Schwingung. Höheres KD = mehr Dämpfung, aber trägere Reaktion. Empfehlung: Immer nur einen Wert gleichzeitig ändern und testen.

BASE_SPEED: Änderung der Grundgeschwindigkeit erfordert möglicherweise Nachjustierung von KP/KD.

Bei neuen Motoren: Alle fünf Asymmetrie-Kompensationsstellen aus Kapitel 4 prüfen.

5.4 Serverprotokoll erweitern

Neuen Befehl hinzufügen: (1) Befehlsstring in `handle_network_command()` ergänzen. (2) Logik in `mission_control.py` implementieren. (3) Format beibehalten: Befehl als Textzeile mit Zeilenumbruch.

Neue Nachricht vom Pico: Via `_queue_server_message()` in die Warteschlange stellen. Serverseitig in `process_incoming_msg()` verarbeiten.

5.5 Pin-Belegung

Die gesamte Pin-Belegung ist ausschließlich in `main.py` konfiguriert:

Komponente	GPIO-Pins
Motor A (Links)	3, 4 (Richtung), 2 (PWM/Geschwindigkeit)

Komponente	GPIO-Pins
Motor B (Rechts)	5, 6 (Richtung), 7 (PWM/Geschwindigkeit)
Taster	0 (Rot), 1 (Grün)
Linien Sensoren	8, 9, 10, 11, 12 (LA, LM, M, RM, RA)
Buzzer	13
Server-LED (RGB)	16, 17, 18 (Rot, Grün, Blau)
Pico-LED (RGB)	19, 20, 21 (Rot, Grün, Blau)
Ultraschall Füllstand	22 (Trigger), 26 (Echo)
Ultraschall Hindernis	27 (Trigger), 28 (Echo)